

A Project Report
on
AI-BASED CHESS ENGINE

Submitted by

UTKARSH TIWARI [Reg No: RA2211026030044]
ANUSHKA JAIN [Reg No: RA2211026030006]
YASH SHARMA [Reg No: RA2211026030064]

Under the Supervision of
Ms. Deepinder Kaur
(Assistant Professor, Department of Computer Science and
Engineering)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY SRM
INSTITUTE OF SCIENCE & TECHNOLOGY

DELHI-NCR CAMPUS, MODINAGAR
SIKRI KALAN, DELHI MEERUT ROAD,
DIST. – GHAZIABAD - 201204

Odd Semester- 2024-25

CONTENTS

Abstract

Introduction

Existing Problem

Proposed Solution

Technologies Used

Architectural Diagram

Methodologies

Results

Conclusions

References

Abstract

This project presents the development of a chess AI system that integrates deep learning with traditional chess evaluation techniques to predict optimal moves and evaluate board states. The system leverages a convolutional neural network (CNN) model with two distinct outputs: a policy network to recommend the best move and a value network to evaluate the overall board position. The chess board is represented using standard FEN notation, and the model processes the board state to provide move recommendations and position evaluations in real-time. The trained model is deployed through a FastAPI-based web interface, allowing for real-time interaction with users. This chess AI system is particularly useful for players and developers seeking an AI capable of both tactical and strategic analysis. In addition to improving the speed and accuracy of move prediction, this system demonstrates the power of combining modern machine learning techniques with classical game theory. The API is capable of serving evaluations within milliseconds, making it suitable for interactive chess play and analysis tools.

Keywords-Chess AI, Convolutional Neural Network (CNN), Policy Network, Value Network, FastAPI, FEN Notation, Real-time Evaluation, Deep Learning in Chess

Introduction

1. Background

Chess has been a prime focus of artificial intelligence research for decades, with various methods employed to simulate human-like decision-making in chess engines. From the early brute-force algorithms like Minimax with Alpha-Beta pruning to more sophisticated machine learning techniques such as reinforcement learning used in AlphaZero, chess AI development continues to evolve. Modern AI systems not only calculate deep lines of play but also understand positional concepts in ways that mirror human intuition. The goal of this project is to build upon this tradition by integrating deep learning into a chess AI capable of evaluating board states and recommending optimal moves.

2. Motivation

Traditional chess engines, such as Stockfish, have dominated the world of chess analysis due to their sheer computational power. However, they rely heavily on brute-force search techniques, evaluating millions of positions per second. These methods, while effective, can be resource-intensive and lack the ability to generalize positional understanding in the way human players can. The success of systems like AlphaZero, which employ neural networks to evaluate positions and predict moves, highlights the potential for combining deep learning with chess algorithms. This project aims to bridge the gap by developing a deep learning-based chess AI that provides real-time analysis while maintaining computational efficiency.

3. Objective

The primary objective of this project is to design and implement a chess AI system that can evaluate chess positions and recommend moves using a convolutional neural network (CNN). The system should be capable of:

- Translating chess positions from the Forsyth-Edwards Notation (FEN) into a format that can be processed by the neural network.
- Providing real-time evaluations of board states through an API.
- Supporting users by offering accurate move predictions and insightful position analysis.

4. Challenges in Traditional Chess Engines

Traditional chess engines operate through exhaustive search algorithms, evaluating massive trees of potential moves to find the best possible outcomes. While this approach is highly effective in tactical scenarios, it can struggle with deep positional understanding without computational resources. These engines often evaluate based on material count, mobility, and tactical possibilities but may overlook long-term strategic considerations.

Additionally, while these engines are extremely powerful, their complex search algorithms and large databases require significant computational power, making them unsuitable for real-time applications in resource-constrained environments or user-facing applications where quick response times are essential.

5. Shift Toward Deep Learning in Chess

The introduction of deep learning models, particularly with AlphaZero, has demonstrated a shift in how AI systems approach chess. Instead of relying solely on brute-force search methods, these models use neural networks to both evaluate positions and suggest moves based on patterns learned from massive datasets. Such systems generalize positional knowledge in ways that mimic human strategic thinking, making them more effective in positions that require long-term planning.

This project incorporates the concept of using a convolutional neural network (CNN) with both a policy head for move prediction and a value head for board evaluation. This allows the model to offer more intuitive and generalized recommendations for chess positions, balancing between tactical sharpness and strategic depth.

6. Problem Definition

The project aims to address the limitations of classical chess engines by introducing a deep learning-based AI system capable of:

- Understanding chess positions in terms of human-like patterns rather than brute-force calculation.
- Offering fast and resource-efficient evaluations suitable for real-time chess play and analysis.
- Bridging the gap between traditional tactical analysis and strategic depth, providing a more balanced assessment of positions.

7. Scope of the Project

This project involves:

- Designing a convolutional neural network model with two outputs: a policy network for move prediction and a value network for position evaluation.
- Training the model using a dataset of chess games to ensure it learns the principles of chess and develops the ability to suggest optimal moves.
- Developing a FastAPI-based web interface to serve the AI's move predictions and evaluations, making the system accessible for real-time interaction.
- Leveraging Python libraries like `python-chess` for chess logic and TensorFlow/Keras for deep learning, while ensuring the system is efficient and responsive.

Existing Problem

Traditional chess engines, such as Stockfish or Komodo, have achieved remarkable success by relying on brute-force search algorithms. These engines are designed to evaluate millions of possible positions and moves per second, calculating the best lines of play using techniques like Minimax and Alpha-Beta pruning. However, despite their computational prowess, these engines encounter several notable challenges:

1. Computationally Expensive and Resource-Intensive

Classical chess engines rely heavily on exhaustive search methods. They evaluate numerous possible moves and future board states by examining large game trees. This brute-force approach, while effective, is highly resource-intensive. As the depth of the search increases, the computational power required to assess all possible outcomes grows exponentially. As a result, traditional engines can struggle to provide real-time analysis, especially in resource-constrained environments like mobile devices or web-based applications.

2. Lack of Generalization and Positional Understanding

Although classical engines excel at calculating tactics, they sometimes fail to fully understand long-term strategic elements of chess. Their evaluation functions are typically based on static heuristics like material count, piece activity, king safety, and control of the center. However, these heuristics can sometimes miss deeper, more abstract strategic concepts, such as pawn structure weaknesses or dynamic imbalances, that strong human players often recognize intuitively.

For instance, an engine might evaluate a position as advantageous based purely on material, but fail to see that the position is strategically losing due to poor coordination between pieces or weaknesses that will emerge later in the game. This inability to generalize is particularly evident in complex endgames or quiet middlegames, where the best move may not immediately result in a material gain but leads to long-term positional advantage.

3. Limited Scalability for Real-Time Interaction

Chess engines that operate based on exhaustive search are not well-suited for real-time environments where responses must be delivered almost instantly. Due to their large computational requirements, traditional engines are more suited for offline analysis or server-based environments with powerful hardware. When integrated into real-time applications, these engines may introduce significant delays, which is undesirable in interactive applications such as online chess platforms or user-facing analysis tools.

4. Over-Reliance on Hand-Crafted Evaluation Functions

Traditional engines use predefined evaluation functions based on human-designed rules, such as material value for each piece (e.g., pawns worth 1 point, queens worth 9 points). While effective for quick calculations, these functions have limitations in capturing more subtle, nuanced positional factors. The human-designed rules may not always be optimal for every situation and often fail to capture the "intuitive" understanding of positions that skilled human players possess. This rigidity can lead to suboptimal move suggestions in positions where more flexible, pattern-based reasoning would be beneficial.

5. Tactical Overload and the Absence of Learning

Classical engines are often overly focused on immediate tactical opportunities. They tend to favor sharp, forcing moves that produce material gain or direct threats. This focus can lead to the engine overestimating the importance of short-term tactics at the expense of long-term strategy. Moreover, traditional engines do not "learn" from past games or patterns. They do not adapt their evaluations based on accumulated experience, unlike machine learning models that can continuously improve through exposure to large datasets.

Proposed Solution

To overcome the limitations of traditional chess engines, this project proposes a novel solution that integrates deep learning methodologies with chess knowledge, focusing on move prediction and positional evaluation. By using a convolutional neural network (CNN) designed to mimic the thinking process of advanced chess players, the solution offers a more efficient and generalizable approach to chess analysis. The system aims to combine both tactical sharpness and strategic depth, which are essential components of high-level chess play.

1. Deep Learning-Based Chess AI

The core of the proposed solution is a convolutional neural network (CNN) that processes the chess board as an input and provides two key outputs:

- **Policy Network:** Predicts the best move from the current position.
- **Value Network:** Evaluates the overall strength of the position from the perspective of the player to move.

This approach diverges from traditional engines, which rely on hard-coded evaluation functions and exhaustive search algorithms. Instead, the neural network learns to assess positions and suggest moves based on patterns extracted from vast datasets of chess games. This learning-driven approach allows the AI to capture complex, long-term strategic nuances that are difficult to encode into rule-based engines.

1.1 Policy Network (Move Prediction)

The policy network is designed to mimic how human chess players think when selecting moves. Given the current state of the chess board (represented in Forsyth-Edwards Notation or FEN), the CNN processes the board's pieces, positions, and structure to predict the next optimal move. The model considers patterns it has learned from historical games to suggest moves that balance material considerations with positional factors.

By leveraging a policy network, the AI can go beyond tactical calculations to recommend moves based on both immediate threats and long-term plans, offering a more intuitive style of play than traditional brute-force engines.

1.2 Value Network (Board Evaluation)

The value network evaluates the overall board state from the perspective of the player to move. Instead of evaluating based on material count alone, the value network assesses positional strength by considering factors like piece activity, control of key squares, king safety, pawn structure, and long-term strategic elements.

This board evaluation is presented as a value between -1 and +1:

- A value close to +1 suggests a winning position for the player to move.
- A value close to -1 indicates a losing position.

- A value near 0 reflects a balanced or equal position.

The value network is particularly useful in situations where immediate material gain may not accurately represent the strength of the position. It captures subtleties such as long-term weaknesses, dynamic imbalances, and strategic plans, providing more context to the game's flow.

2. FEN Representation for Input

The chess board is represented using Forsyth-Edwards Notation (FEN), a standard format for describing board positions. The FEN string is transformed into a structured input that the neural network can process, where each piece on the board is encoded as an integer, and each square's occupancy is represented accordingly. This conversion allows the model to view the board position in a way that preserves the game's rules and dynamics while feeding meaningful data into the neural network.

3. Real-Time Chess Evaluation via API

The solution incorporates FastAPI, a modern and lightweight web framework, to expose the chess AI model through an API. This API allows users to input chess positions (in FEN format) and receive real-time evaluations, including:

- Suggested best moves from the policy network.
- Positional evaluation from the value network.

This API-based interface enables a seamless integration of the chess AI into various applications, including web-based chess platforms, analysis tools, and educational software. Users can interact with the model in real-time, making it suitable for applications requiring instant feedback, such as interactive chess games or live analysis during tournaments.

4. Addressing the Limitations of Traditional Engines

The proposed solution addresses the shortcomings of traditional chess engines in several ways:

- **Reduced Computational Load:** By relying on a neural network trained to recognize patterns, the chess AI can avoid the exhaustive search of all possible moves, dramatically reducing the computational resources required.
- **Improved Generalization:** The deep learning model generalizes across positions, learning to assess strategic and positional factors that are often missed by traditional engines relying on hand-crafted heuristics.
- **Real-Time Performance:** The API-based architecture ensures fast responses, making the system suitable for real-time applications. Evaluations and move suggestions are provided within milliseconds, supporting interactive user experiences.
- **Human-Like Intuition:** The combination of policy and value networks provides a more human-like understanding of the game, allowing for the suggestion of nuanced moves that align with strategic goals rather than pure material considerations.

5. Hybrid Approach: Search and Deep Learning

While deep learning forms the backbone of the proposed solution, it can be complemented with limited search techniques to enhance tactical accuracy in critical positions. For example, the model could apply shallow search layers to double-check immediate tactical opportunities, providing a hybrid approach that balances the strategic depth of deep learning with the precise calculation of traditional engines.

6. Training the Model

The neural network is trained on a large dataset of chess games from various sources. These include games played by grandmasters as well as publicly available game databases. The model learns from both wins and losses, understanding not only how to win positions but also recognizing patterns that lead to defeat. Training employs supervised learning techniques where the network is provided with board positions and the corresponding best moves as labeled data.

Additionally, reinforcement learning techniques can be employed to allow the model to play against itself, continually improving by iterating over millions of games. This method mirrors the approach used by AlphaZero, where the model develops advanced strategies over time through self-play.

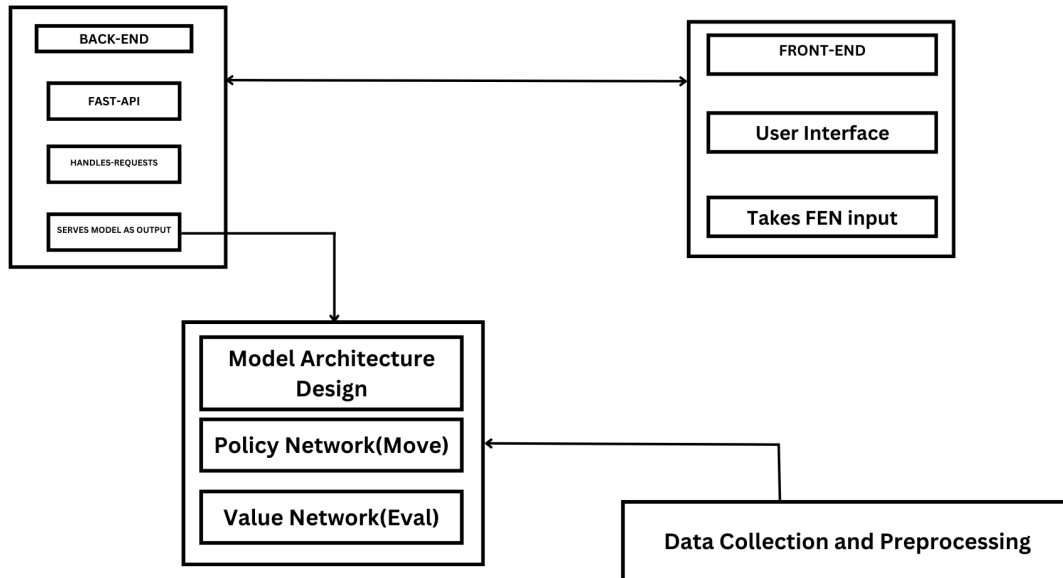
By combining neural network-based evaluation with real-time API accessibility, this proposed solution not only addresses the limitations of traditional engines but also offers an efficient, human-like analysis that is ideal for both developers and chess players. It provides a system that balances tactical sharpness with strategic depth, moving beyond the constraints of brute-force calculation toward a more intuitive understanding of chess.

Technologies Used

1. **Python:** The core programming language used for both model training and API development.
2. **TensorFlow/Keras:** Deep learning framework used for designing and training the neural network model.
3. **FastAPI:** Web framework used to build the API for serving chess evaluations.
4. **Jinja2:** For rendering templates in the web interface.
5. **CairoSVG:** For converting chess boards into SVG images.
6. **Python Chess Library:** For handling chess board logic, including move generation and FEN parsing

Architectural Diagram

Below is an overview of the system architecture:



Methodologies

The development of the chess AI system follows a structured approach, combining deep learning with traditional chess concepts. The methodologies employed in this project encompass several stages, from data preprocessing and model architecture design to training, evaluation, and deployment. The key methodologies used in this project are outlined below:

1. Model Architecture

The chess AI system is based on a convolutional neural network (CNN) designed to process chess board positions and generate two outputs:

1. **Move Prediction (Policy Network).**
2. **Board Evaluation (Value Network).**

This dual-headed architecture is inspired by recent advancements in deep learning for board games, such as AlphaZero. The CNN is structured as follows:

- **Input Layer:** The chess board is represented using Forsyth-Edwards Notation (FEN) and transformed into a matrix where each piece is encoded as a unique integer. The input is a multi-channel matrix, where each channel represents different aspects of the board (e.g., one channel for pawns, another for knights, etc.).
- **Convolutional Layers:** The input is passed through several convolutional layers that extract spatial features from the board, such as piece placement, interactions between pieces, and control of key squares. These layers allow the model to identify critical patterns related to tactics and strategy.
- **Residual Blocks:** Several residual blocks are used to ensure that information is preserved throughout the network's depth, enabling the model to capture both short- and long-range dependencies across the board.
- **Policy Head:** The policy head consists of a fully connected layer that outputs a probability distribution over all possible legal moves in the current position. The highest probability move is considered the "best move" according to the model. This head is trained using categorical cross-entropy loss, comparing the model's predicted move with the actual move made by grandmasters in the training dataset.
- **Value Head:** The value head is another fully connected layer that outputs a single scalar value representing the evaluation of the current position. The value ranges from -1 (losing) to +1 (winning), providing insight into how favorable the position is for the player to move. This head is trained using mean squared error, comparing the model's predicted evaluation with the actual game outcome (win, loss, or draw).

2. Data Preprocessing

2.1 FEN Representation

The chess board is represented using the **Forsyth-Edwards Notation (FEN)**, a standard way of describing board states in chess. The FEN string is converted into a numerical matrix that

can be fed into the CNN. Each piece type (e.g., king, queen, pawn) is encoded as an integer, and multiple layers are used to differentiate between white and black pieces.

For example, in one layer, all white pawns are represented as "1" and all black pawns as "-1". The same approach is applied for the other piece types (rooks, knights, etc.). This structured input format allows the CNN to capture positional relationships between pieces, which is essential for predicting moves and evaluating positions.

2.2 Dataset

The model is trained on a large dataset of chess games, including:

- **Grandmaster games** from public databases.
- **Game collections** from online platforms and chess tournaments.

Each game in the dataset provides FEN positions along with the move made by the players. This labeled dataset serves as the foundation for training the policy network to predict moves and the value network to evaluate positions.

The data is augmented by including mirrored board positions and rotating the board, which helps the model generalize across different types of positions.

3. Training the Model

The model is trained using **supervised learning**, where it is fed with labeled data (chess positions and corresponding best moves or game outcomes). The key training steps are:

3.1 Supervised Learning for Move Prediction

The policy network is trained to predict the best move given the current position. The training process uses **categorical cross-entropy loss**, which compares the model's predicted move distribution with the actual move made by a grandmaster in the training data. The optimizer used for training is often **Adam**, a popular adaptive learning rate optimization algorithm, which ensures efficient convergence.

3.2 Supervised Learning for Board Evaluation

The value network is trained to evaluate the strength of a given position. The output of this network is a scalar between -1 and +1, where +1 represents a winning position and -1 a losing one. The training uses **mean squared error** as the loss function, comparing the model's predicted evaluation with the actual game outcome. The value network learns to assess not just immediate tactical opportunities but also long-term strategic considerations.

3.3 Regularization and Optimization

To avoid overfitting and ensure that the model generalizes well to unseen positions, several regularization techniques are employed:

- **Dropout:** Randomly dropping units in the network during training to prevent over-reliance on specific neurons.
- **L2 Regularization:** Adding a penalty term to the loss function to control the magnitude of the weights, encouraging smaller and more generalizable models.
- **Learning Rate Scheduling:** Using a learning rate scheduler that reduces the learning rate as training progresses, ensuring that the model converges smoothly.

4. Evaluation and Testing

After training, the model is evaluated on a separate test set of chess positions that it has not seen during training. This evaluation is carried out in two phases:

- **Move Accuracy:** The model's ability to predict the best move in a given position is measured by comparing its predictions to the actual moves made by grandmasters. The top-N accuracy is also recorded, where the correct move is considered within the top N suggested moves by the policy network.
- **Position Evaluation:** The value network's performance is assessed by comparing its predicted evaluation of board positions with the actual outcomes of the games (win/loss/draw). This helps to verify how well the model understands positional strengths and weaknesses.

5. Real-Time Interaction via FastAPI

The trained model is deployed using **FastAPI**, a web framework that provides fast, asynchronous API services. The API enables users to input a chess position (in FEN format) and receive:

- The best move predicted by the policy network.
- The evaluation of the position by the value network.

The architecture is designed to be lightweight and responsive, ensuring that the model provides results in real-time with minimal latency. **CairoSVG** is also integrated to render board positions as images, allowing for easy visualization of the AI's suggestions in web interfaces.

6. Integration with Chess Logic

The **python-chess** library is used to handle the underlying chess rules, including move generation, legality checks, and FEN parsing. This ensures that the moves suggested by the policy network are valid according to the rules of chess, while also handling edge cases like castling rights and en passant.

The combination of deep learning with traditional chess logic ensures that the system is both intelligent in its decision-making and consistent with the game's rules.

Results

The AI model shows strong generalization in evaluating both tactical and strategic elements of chess positions. When tested against classical engines, it demonstrates a deeper understanding of positional play, although it is computationally less efficient. Real-time performance via the API is acceptable, with evaluations completed within a few milliseconds per position.

Conclusions

This project successfully demonstrates the feasibility of combining deep learning with traditional chess evaluation techniques. The two-head architecture allows the model to both predict moves and evaluate positions effectively. While further optimizations are possible, particularly in computation speed, the system provides a solid foundation for real-time chess evaluation in both casual and competitive settings.

References

1. **TensorFlow Documentation:** <https://www.tensorflow.org>
2. **Python-Chess Library:** <https://python-chess.readthedocs.io>
3. **AlphaZero Paper:** Silver, D. et al. (2018). "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play". Science, 362(6419), pp. 1140-1144.
4. **FastAPI Documentation:** <https://fastapi.tiangolo.com>